BÀI THỰC HÀNH 7. KIỂM THỬ MOCKING ỨNG DỤNG DI ĐỘNG VÀ TÍCH HỢP LIÊN TỤC (CI/CD)

(4 tiết)

I. Mục tiêu

Sau khi hoàn thành bài thực hành này, sinh viên có khả năng:

- **Phân tích và thiết kế kịch bản kiểm thử trên giấy:** Tự tay xây dựng bảng ma trận ca kiểm thử (Test Matrix) cho các dịch vụ ngoại vi trước khi bắt tay vào lập trình.
- **Lập trình kiểm thử Mocking di động:** Áp dụng thư viện `unittest.mock.patch` để đóng thế phần cứng GPS (`get_current_location`) và giả lập sự cố kết nối mạng di động (`ConnectionError`) mà không phụ thuộc vào thiết bị thật.
- **Xây dựng đường ống CI tự động với GitHub Actions:** Lập trình tập tin cấu hình YAML để thiết lập môi trường máy chủ sạch (Ubuntu runner), cài đặt dependencies đã ghim phiên bản và tự động kích hoạt bộ test khi có sự kiện `push` hoặc `pull_request`.
- **Thiết lập và vượt qua Cổng chất lượng (Quality Gates):** Trải nghiệm thực tế tình huống đường ống CI bị chặn đỏ khi áp đặt ngưỡng kiểm định (`-fail-under=80`); từ đó sử dụng bảng báo cáo độ bao phủ mã nguồn để tự viết bổ sung các test case còn thiếu nhằm đưa hệ thống trở lại trạng thái Pass.
- **Trải nghiệm phản xạ "Break the Build" & Phục hồi:** Nhận diện ngay lập tức các lỗi cú pháp thông qua Linter (`flake8`) trước khi chạy test, nắm vững nguyên lý thất bại sớm (*Fail-Fast*).
- **Quản trị phiên bản & Trích xuất bằng chứng (Logs & Artifacts):** Rèn luyện tác phong kỹ sư chuyên nghiệp thông qua việc thu thập các log thực thi nguyên bản (*Raw Logs*), lịch sử Git và báo cáo độ phủ HTML.

II. Kiến thức cần nắm

Sinh viên cần nắm vững các nội dung sau:

1. **Kiểm thử di động & Giả lập dịch vụ ngoại vi:** Kỹ thuật Mocking/Stubbing giúp cô lập module cần kiểm thử, loại bỏ sự phụ thuộc vào sóng viễn thông, tín hiệu GPS hoặc máy chủ bên thứ ba.
2. **Tích hợp liên tục (CI) & Vòng phản hồi nhanh (*Fast Feedback Loop*):** Cơ chế phát hiện lỗi sớm bằng việc tự động build và chạy test trên máy chủ ngay sau mỗi commit.
3. **Cổng chất lượng (Quality Gate) & Break the Build:** Nguyên tắc khóa nhánh merge và báo lỗi toàn hệ thống nếu mã nguồn không vượt qua các tiêu chí đo lường (Linter sạch, Độ bao phủ mã nguồn $\geq 80\%$).
4. **Cấu trúc GitHub Actions Workflow:** Quy ước cấu hình YAML gồm `on` (trigger), `jobs`, `runs-on`, `steps`, `uses` (actions) và `run` (shell scripts).
5. **Liêm chính học thuật trong Kiểm thử (Verification of Work):** Hiểu rõ ý nghĩa của việc lưu giữ Log thực thi và dấu vết commit (*Git Traceability*) để chứng minh quá trình làm việc độc lập.

III. Bối cảnh dự án và Phân bổ thời lượng

Bối cảnh dự án thực tế (`YummyApp Mobile Backend`): Bạn là Kỹ sư Lập trình Kiểm thử (Software Engineer in Test – SET) chịu trách nhiệm đảm bảo chất lượng cho backend ứng dụng đặt món ăn `YummyApp`. Ứng dụng có các tính năng cốt lõi:

1. Tính toán tổng giá trị đơn hàng theo số lượng món ăn trong giỏ hàng.
2. Kiểm tra và áp dụng mã khuyến mãi (gọi API xác thực mạng di động, kiểm tra phiên bản HĐH).
3. Đề xuất nhà hàng lân cận (gọi dịch vụ định vị phần cứng GPS của thiết bị).

Phân bổ thời lượng 4 tiết (200 phút):

- **Tiết 1 (50 phút):** Khởi tạo dự án, chạy bộ test cơ bản, xuất log terminal cá nhân và đẩy code lên GitHub.
- **Tiết 2 (50 phút):** Vẽ tay Sơ đồ kiến trúc CI → Lập trình file YAML cơ bản chạy thành công.
- **Tiết 3 (50 phút):** Kích hoạt Cổng chất lượng $\geq 80\%$ (CI báo Đỏ) → Viết tay Bảng thiết kế Test → Tự tay lập trình bộ test Mocking để đưa CI về Xanh → Viết tay trả lời 3 câu hỏi phân tích.
- **Tiết 4 (50 phút):** Thực hành Break the Build → Xuất HTML Artifacts → Xuất trọn bộ 3 file Log thực thi → Viết tay trả lời 3 câu hỏi Version Pinning.

IV. Hướng dẫn thực hành và Yêu cầu minh chứng

CẤP ĐỘ 1 (DỄ – TIẾT 1 & 2): DỰ ÁN CƠ BẢN VÀ THIẾT LẬP CI PIPELINE

1. Nhiệm vụ Tiết 1 — Chuẩn bị mã nguồn và trích xuất Log kiểm thử cục bộ:

1. Tạo thư mục dự án `yummyapp-cicd/` và thư mục con `logs/`.
2. Sao chép 3 tập tin mã nguồn ở Mục VI vào thư mục dự án: `gps_service.py`, `yummy_app.py` và `test_yummy_app.py`.
3. Tạo tập tin `requirements.txt` với danh sách thư viện đã ghim phiên bản chuẩn:

```
requests==2.31.0
coverage==7.4.0
flake8==7.0.0
```

4. Cài đặt dependencies (`pip install -r requirements.txt`).
5. **Yêu cầu xuất Log 1 (`local_test.log`):** Thực hiện chạy kiểm thử cục bộ và trích xuất kết quả ra file log bằng lệnh terminal sau:

```
coverage run -m unittest discover -s . -p "test_*.py" > logs/
   local_test.log 2>&1
coverage report -m >> logs/local_test.log
```

Kiểm tra tập tin `logs/local`$_{t}est.logØØmboktquchytrnmycnhnØØcghiliØyØ$.

5. `Khởi tạo Git cục bộ (git init), cấu hình định danh cá nhân:`

```
git config user.name "MSSV - Ho Va Ten"
git config user.email "mssv@dlu.edu.vn"
```

Thực hiện commit đầu tiên và liên kết với kho lưu trữ công khai trên GitHub của bạn.

2. Nhiệm vụ Tiết 2 — Vẽ tay Sơ đồ CI và Lập trình đường ống cơ bản:

1. **YÊU CẦU VIẾT TAY 1 (Sơ đồ luồng CI Pipeline):** Trên một trang giấy trắng, dùng bút tự tay vẽ sơ đồ khối kiến trúc của đường ống CI với đầy đủ 6 khối: *Git Push Event* $\rightarrow$ *Checkout Code* $\rightarrow$ *Setup Python 3.11* $\rightarrow$ *Install Dependencies* $\rightarrow$ *Execute Unit Tests* $\rightarrow$ *Quality Gate*. Ở góc trên trang giấy, bắt buộc ghi rõ: **Họ và tên, MSSV, Ngày thực hành và Chữ ký tươi**.
2. Tạo tập tin cấu hình `.github/workflows/yummyapp-ci.yml` trong dự án:

```
name: YummyApp Mobile Backend CI Pipeline

on:
  push:
    branches: [ <DIEN_TEN_NHANH_CHINH> ]     # Goi y: main
  pull_request:
    branches: [ <DIEN_TEN_NHANH_CHINH> ]     # Goi y: main

jobs:
  mobile-test-job:
    runs-on: <DIEN_TEN_RUNNER_UBUNTU>        # Goi y: ubuntu-
       latest

    steps:
      - name: 1. Checkout Source Code
        uses: actions/checkout@v4

      - name: 2. Set up Python Environment
        uses: actions/setup-python@v5
        with:
          python-version: '<DIEN_PHIEN_BAN>' # Goi y: '3.11'

      - name: 3. Install Dependencies
        run: |
          python -m pip install --upgrade pip
          pip install -r <DIEN_TEN_FILE_REQUIREMENTS>

      - name: 4. Execute Test Suite
        run: |
          <DIEN_LENH_CHAY_UNITTEST_DISCOVER>
```

3. Commit và push tập tin lên GitHub. Truy cập tab **Actions**, kiểm tra và chụp ảnh màn hình Workflow chạy thành công (Tích xanh **Pass**).

CẤP ĐỘ 2 (TRUNG BÌNH – TIẾT 3): THIẾT KẾ TEST TRÊN GIẤY & VƯỢT QUALITY GATE

1. Kích hoạt Cổng chất lượng trên CI (Quan sát Build bị chặn Đỏ): Cập nhật tập tin `.github/workflows/yummyapp-ci.yml`, thêm bước Linter và ngưỡng Coverage $\geq 80\%$:

```
      # Buoc kiem tra cu phap va loi tinh
      - name: 4. Static Code Analysis (Linter)
        run: |
          flake8 . --count --select=E9,F63,F7,F82 --show-source --
              statistics

      # Buoc do luong Coverage va khoa cong duoi 80%
      - name: 5. Execute Tests & Quality Gate
        run: |
          coverage run -m unittest discover -s . -p "test_*.py"
          coverage report -m
          coverage html -d coverage_html
          coverage report --fail-under=80
```

Push lên GitHub và ghi nhận: Workflow bị **FAILED (Báo Đỏ)** tại bước 5 do độ bao phủ mã nguồn ban đầu chỉ đạt $\approx 35\%$, không đạt ngưỡng 80%.

2. Thiết kế kịch bản kiểm thử trên giấy & Lập trình Mocking:

1. **YÊU CẦU VIẾT TAY 2 (Bảng thiết kế kịch bản Test Case):** Trước khi viết mã, sinh viên kẻ một bảng trên giấy trắng (kèm Họ tên, MSSV, Chữ ký) liệt kê 7 kịch bản kiểm thử cần bổ sung:
   - Bảng gồm 4 cột: *Tên Test Case* | *Hàm được gọi* | *Đối tượng & Giá trị Mocking* | *Kết quả kỳ vọng (Assert)*.
   - 4 kịch bản cho `apply_promo_code`: Online hợp lệ, Offline ném `ConnectionError`, Mã đặc biệt trên OS 15, Mã đặc biệt trên OS 14 ném `ValueError`.
   - 3 kịch bản cho `get_nearby_restaurants`: Mock GPS trong vùng `(10.0, 105.0)`, Mock GPS ngoài vùng `(20.0, 110.0)`, Mock GPS ném `GPSSignalLostError`.
2. **Lập trình mã nguồn kiểm thử:** Mở `test_yummy_app.py`, dựa vào bảng thiết kế viết tay ở trên, tự tay lập trình hoàn chỉnh 2 lớp `TestMediumLevel` và `TestHardLevel` (sử dụng `@patch('yummy_app.gps_service.get_current_location')`).

3. YÊU CẦU VIẾT TAY 3 (Trả lời câu hỏi lý thuyết Cấp độ 2): Trên giấy trắng (ghi rõ Họ tên, MSSV, Chữ ký), sinh viên tự tay viết câu trả lời cho 3 câu hỏi sau:

- *Câu 2.1:* Tại sao bước chạy Linter (`flake8`) phải đặt trước bước chạy test trong đường ống CI?
- *Câu 2.2:* Tham số `-fail-under=80` của lệnh `coverage report` hoạt động theo nguyên lý nào trong hệ điều hành Linux để báo hiệu cho GitHub Actions dừng tiến trình build?
- *Câu 2.3:* Tham số `-m` trong lệnh `coverage report -m` hiển thị thêm thông tin gì và giúp ích gì cho kỹ sư kiểm thử khi tối ưu độ bao phủ mã nguồn?

CẤP ĐỘ 3 (NÂNG CAO – TIẾT 4): TRÍCH XUẤT RAW LOGS VÀ QUẢN TRỊ PHIÊN BẢN

1. **Thực hành Break the Build có chủ đích:** Cố tình tạo một hàm sai cú pháp trong `yummy_app.py`, push lên GitHub để chứng minh bước Linter bắt được lỗi ngay từ đầu. Sau đó xóa lỗi để đưa build trở lại màu Xanh.

2. **Cấu hình xuất Artifacts HTML Coverage Report:** Bổ sung step sau vào cuối file YAML:

```
      - name: 6. Upload HTML Coverage Report Artifact
        if: always()
        uses: actions/upload-artifact@v4
        with:
          name: YummyApp-Coverage-Report
          path: coverage_html/
          retention-days: 7
```

3. **Yêu cầu xuất Log 2 (`git_history.log`):** Chạy lệnh terminal sau để xuất toàn bộ cây lịch sử commit có gắn mác định danh sinh viên:

```
git log --graph --pretty=format:"%h - %an (%ad): %s" > logs/
   git_history.log
```

4. **Yêu cầu xuất Log 3 (`github_actions_run.log`):** Trên giao diện GitHub Actions, mở lần chạy workflow thành công cuối cùng → bấm vào menu ba chấm góc phải → chọn **View raw logs** (hoặc *Download log archive*) → lưu nội dung file log vào $\texttt{logs/github}_{a}\texttt{ctions}_{r}\texttt{un.log}$.

4. YÊU CẦU VIẾT TAY 4 (Trả lời câu hỏi lý thuyết Cấp độ 3): Viết tay câu trả lời cho 3 câu hỏi sau lên giấy (kèm Họ tên, MSSV, Chữ ký):

   - *Câu 3.1:* Phân biệt giữa khái niệm Flaky Test và lỗi do sai lệch môi trường phụ thuộc.
   - *Câu 3.2:* Tại sao trên máy chủ CI bắt buộc phải tuân thủ nguyên tắc ghim phiên bản (Version Pinning) trong requirements.txt?
   - *Câu 3.3:* Trình bày quy trình 6 bước chuẩn để nâng cấp một thư viện lên phiên bản mới an toàn qua Pull Request mà không làm gián đoạn nhánh main.

V. Yêu cầu nộp bài & Quy định Liêm chính Học thuật

1. Cấu trúc thư mục nộp bài (`MSSV_HoTen_Lab7.zip`): Sinh viên nén toàn bộ thư mục bài làm thành 01 file zip duy nhất theo cấu trúc bắt buộc sau:

```
MSSV_HoTen_Lab7/
|-- .github/workflows/yummyapp-ci.yml   <- File YAML CI hoan chinh
|-- gps_service.py                      <- File ma nguon phan cung GPS
|-- yummy_app.py                        <- File ma nguon nghiep vu
|-- test_yummy_app.py                   <- Bo test sinh vien tu viet
|-- requirements.txt                    <- Danh sach thu vien da ghim
|-- logs/                               <- THU MUC CHUA 3 FILE LOG MINH
   CHUNG
|   |-- local_test.log                  <- Log chay test va coverage
   tren may
|   |-- git_history.log                 <- Log lich su commit co MSSV
|   |-- github_actions_run.log          <- Raw log tai ve tu GitHub
   Actions
|-- MinhChung_VietTay_MSSV.pdf          <- FILE SCAN/ANH CHUP CAC TRANG
   VIET TAY
|-- BaoCao_KetQua_Lab7.pdf              <- File bao cao chua link GitHub
    & anh chup
```

2. Danh mục các trang viết tay bắt buộc trong `MinhChung_VietTay_MSSV.pdf`**:**

- **Trang 1:** Sơ đồ luồng kiến trúc CI Pipeline (Vẽ tay hoàn toàn).
- **Trang 2:** Bảng ma trận thiết kế kịch bản Test Case Mocking (Kẻ bảng và viết tay).
- **Trang 3 & 4:** Lời giải viết tay chi tiết cho 6 câu hỏi phân tích (Câu 2.1–2.3 và Câu 3.1–3.3).
- **Quy tắc xác thực:** *Mỗi trang viết tay bắt buộc phải có Họ tên, MSSV, Ngày làm bài và Chữ ký tươi bằng bút mực xanh ở góc trên hoặc đặt thẻ sinh viên bên cạnh khi chụp hình.*

3. QUY ĐỊNH LIÊM CHÍNH HỌC THUẬT (CHỐNG LẠM DỤNG AI & GIAN LẬN):

- Mọi bài nộp sử dụng văn bản gõ máy cho phần lý thuyết hoặc sử dụng câu trả lời sinh tự động từ AI mà không có bản viết tay chính chủ kèm chữ ký sẽ bị **0 điểm** toàn bộ phần lý thuyết và thiết kế.
- Thiếu 01 trong 3 file log chứng thực (`local_test.log`, `git_history.log`, `github_actions_run.log`) hoặc thông tin tác giả commit không khớp với MSSV nộp bài sẽ bị coi là sao chép bài của người khác.

VI. Mã nguồn khởi tạo ban đầu phát cho sinh viên

1. Tập tin `gps_service.py`

```
class GPSSignalLostError(Exception):
    """Ngoai le tuy chinh khi thiet bi mat tin hieu dinh vi GPS."""
    pass


def get_current_location():
    """
    Ham mo phong truy cap phan cung GPS dien thoai de lay toa do.
    Trong moi truong kiem thu tu dong, ham nay se duoc dong the (mock).
    """
    return (10.0, 105.0)
```

2. Tập tin `yummy_app.py`

```
import gps_service

MENU = {
    "pho": 50000,
    "bun": 45000,
    "com": 30000,
}

PROMO_CODES = {
    "GIAM10": {"discount": 0.1, "min_os_version": 0},
    "NEW_OS_ONLY": {"discount": 0.2, "min_os_version": 15},
}

RESTAURANTS_BY_LOCATION = {
```

```
    "10.0,105.0": ["Nha hang Pho Ly Quoc Su", "Bun cha Huong Lien"],
}


class YummyApp:
    def __init__(self, network_status="online", os_version=15):
        self.network_status = network_status
        self.os_version = os_version

    def calculate_order_total(self, cart: dict) -> float:
        """Tinh tong gia tri don hang dua tren gio hang."""
        total = 0.0
        for item, quantity in cart.items():
            if item in MENU and quantity > 0:
                total += MENU[item] * quantity
        return total

    def apply_promo_code(self, total: float, code: str) -> float:
        """Kiem tra va ap dung ma khuyen mai."""
        if self.network_status == "offline":
            raise ConnectionError("Khong co ket noi mang de xac thuc ma.
               ")
        if code not in PROMO_CODES:
            raise ValueError("Ma khuyen mai khong hop le.")
        promo = PROMO_CODES[code]
        if self.os_version < promo["min_os_version"]:
            raise ValueError(
                f"Ma nay yeu cau HDH phien ban {promo['min_os_version']}
                    tro len."
            )
        return total * (1 - promo["discount"])

    def get_nearby_restaurants(self):
        """Goi dich vu GPS de de xuat nha hang lan can."""
        try:
            lat, lon = gps_service.get_current_location()
            location_key = f"{lat},{lon}"
            return RESTAURANTS_BY_LOCATION.get(location_key, [])
        except gps_service.GPSSignalLostError:
            return "Khong the xac dinh vi tri."
```

3. Tập tin `test_yummy_app.py` (Bộ test khởi đầu ban đầu)

```
import unittest
from yummy_app import YummyApp
import gps_service


# ============================================================
# CAP DO 1: Bo test co ban cho san (Moi chi dat ~35% Coverage)
# ============================================================
class TestEasyLevel(unittest.TestCase):
    """Kiem thu chuc nang co ban tinh tien don hang"""
    def setUp(self):
        self.app = YummyApp()

    def test_calculate_total_for_single_item(self):
```

```
        cart = {"pho": 2}
        self.assertEqual(self.app.calculate_order_total(cart), 100000)

    def test_calculate_total_for_multiple_items(self):
        cart = {"pho": 1, "bun": 1}
        self.assertEqual(self.app.calculate_order_total(cart), 95000)

    def test_calculate_total_for_empty_cart(self):
        self.assertEqual(self.app.calculate_order_total({}), 0.0)

    def test_calculate_total_with_zero_quantity(self):
        cart = {"pho": 0}
        self.assertEqual(self.app.calculate_order_total(cart), 0.0)


# ================================================================
# CAP DO 2 (TIET 3): SINH VIEN TU TAY LAP TRINH TEST CASE DUOI DAY
# (Sau khi da hoan thanh Ban thiet ke test case viet tay tren giay)
# ================================================================

# Sinh vien tu viet class TestMediumLevel o day (4 test case cho
    apply_promo_code)
# ...

# Sinh vien tu viet class TestHardLevel o day (@patch cho
    get_nearby_restaurants)
# ...


if __name__ == '__main__':
    unittest.main()
```